

# Optimisation des transitions d'un système d'ascenseurs à l'aide des réseaux de Petri

BEN YOUSSEF Yassine

Juin 2025

- 1 Introduction au problème
- 2 Fondements théoriques
- 3 Simulation et résultats
- 4 Conclusion et perspectives

1 Introduction au problème

2 Fondements théoriques

3 Simulation et résultats

4 Conclusion et perspectives

# Introduction

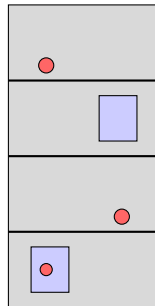
La mauvaise organisation entraîne :

- Des **temps d'attente** longs
- Des **déplacements** inutiles
- Une **surconsommation d'énergie**



# Problématique

Comment optimiser le fonctionnement d'un système d'ascenseurs pour minimiser le temps d'attente des usagers tout en réduisant la consommation énergétique ?



1 Introduction au problème

2 Fondements théoriques

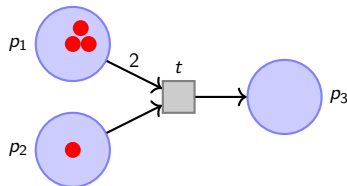
3 Simulation et résultats

4 Conclusion et perspectives

## Définition formelle :

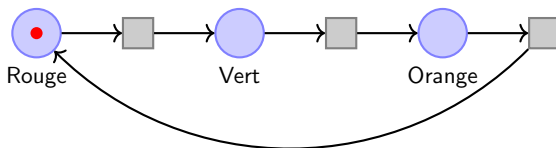
Un réseau de Petri est un 5-uplet  $(P, T, F, W, M_0)$  où :

- $P$  est l'ensemble des places
- $T$  est l'ensemble des transitions
- $F \subseteq (P \times T) \cup (T \times P)$  est l'ensemble des arcs
- $W : F \rightarrow \mathbb{N}^+$  est la fonction de poids des arcs
- $M_0 : P \rightarrow \mathbb{N}$  est le marquage initial



# Exemple : feu tricolore

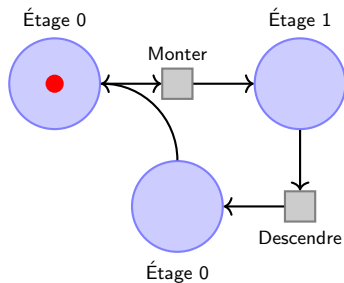
- Système séquentiel simple
- Succession d'états :  
Rouge  $\rightarrow$  Vert  $\rightarrow$   
Orange  $\rightarrow$  Rouge
- Un jeton circule entre les places





# Modèle simplifié à un ascenseur

- Un seul ascenseur dans un immeuble à 2 étages



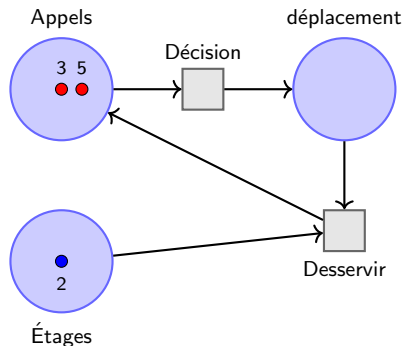
# Modèle avancé avec réseaux de Petri colorés

## Description du modèle :

- **Jetons rouges** : appels en attente avec leur étage
- **Jeton bleu** : position actuelle de l'ascenseur
- La transition **Décision** implémente l'algorithme d'ordonnancement
- La transition **Desservir** simule le service d'un étage

## Avantages des RdP colorés :

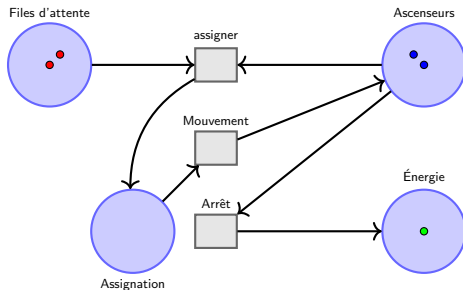
- Les jetons portent des informations :
  - ▶ Étage d'origine de la demande
  - ▶ Étage de destination
  - ▶ Direction souhaitée



# Modélisation complète du système multi-ascenseurs

## Types de jetons utilisés :

- ASCENSEUR = {id, position, direction, capacité}
- APPEL = {origine, destination, temps\_attente}
- ÉNERGIE = {consommation, état}



1 Introduction au problème

2 Fondements théoriques

3 Simulation et résultats

4 Conclusion et perspectives

# Algorithmes classiques d'ordonnement de disque

- **FCFS (First Come First Served)**
  - ▶ Traite les requêtes dans l'ordre exact de leur arrivée
- **SCAN (Algorithme de l'ascenseur)**
  - ▶ Traite les requêtes dans l'ordre du balayage
- **LOOK (Variante de SCAN)**
  - ▶ Similaire à SCAN mais inverse la direction dès qu'il n'y a plus de requêtes

# Algorithme optimisé en énergie et temps d'attribution

```
1: procedure ETAP(appels, ascenseurs)
2:   for chaque nouvel appel  $a$  do
3:      $score_{min} \leftarrow \infty$ 
4:      $asc_{optimal} \leftarrow \text{null}$ 
5:     for chaque ascenseur  $e$  do
6:        $distance \leftarrow |position(e) - origine(a)|$ 
7:        $detour \leftarrow \text{calculDetour}(e, a)$ 
8:        $energie \leftarrow \text{estimationEnergie}(e, a)$ 
9:        $temps \leftarrow \text{estimationTemps}(e, a)$ 
10:       $score \leftarrow \alpha \cdot temps + \beta \cdot energie + \gamma \cdot detour$ 
11:      if  $score < score_{min}$  then
12:         $score_{min} \leftarrow score$ 
13:         $asc_{optimal} \leftarrow e$ 
14:      end if
15:    end for
16:    assigner  $a$  à  $asc_{optimal}$ 
```

# Paramètres de simulation

- Un immeuble de 10 étages équipé de 4 ascenseurs
- Une capacité de 8 personnes par ascenseur
- Des paramètres énergétiques calibrés selon des données de constructeurs d'ascenseurs
- Une période de simulation de 24 heures avec variations de trafic

# Profils de trafic simulés

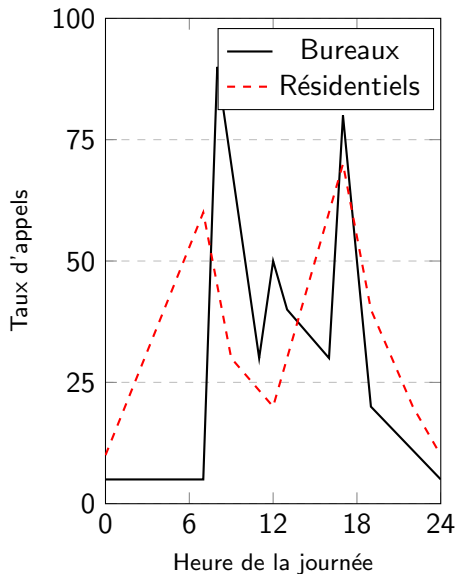
## Profils implémentés :

### • Bureaux :

- ▶ Flux montant le matin (arrivées)
- ▶ Flux descendant le soir (départs)
- ▶ Pics aux heures de repas

### • Résidentiels :

- ▶ Flux descendant le matin
- ▶ Flux montant en fin de journée
- ▶ Distribution plus étalée





## Consommation estimée :

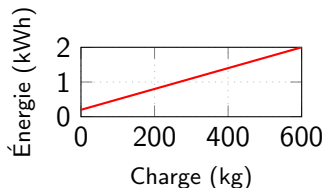
$$E_{total} = \sum_{i=1}^n (E_{deplacement,i} + E_{arret,i})$$

$$E_{deplacement} = k_1 \cdot distance \cdot (m_{cab} + m_{pass})$$

$$E_{arret} = k_2 \cdot nombre\_arrets$$

où :

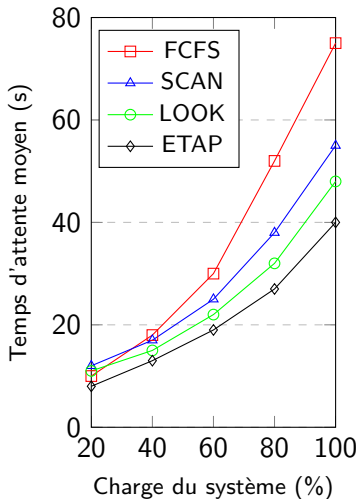
- $m_{cab}$  : masse de la cabine
- $m_{pass}$  : masse des passagers
- $k_1, k_2$  : constantes d'énergie



## Stratégies d'optimisation énergétique :

- **Mode veille** pour les ascenseurs peu utilisés
- **Récupération d'énergie** lors des descentes

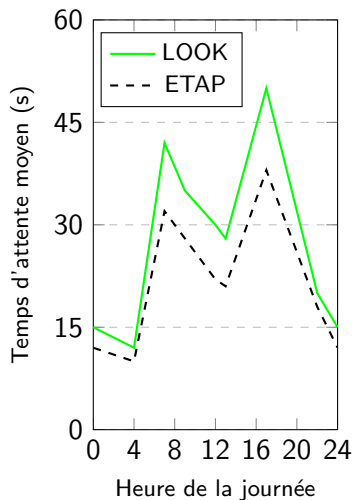
# Résultats : le temps d'attente en fonction de la charge



## Observations :

- L'algorithme ETAP montre une réduction du temps d'attente moyen de **25%** par rapport à LOOK
- Les performances de FCFS se dégradent rapidement avec l'augmentation de la charge
- SCAN et LOOK offrent une amélioration significative par rapport à FCFS
- ETAP maintient un avantage constant à tous les niveaux de charge

# Résultats : le temps d'attente



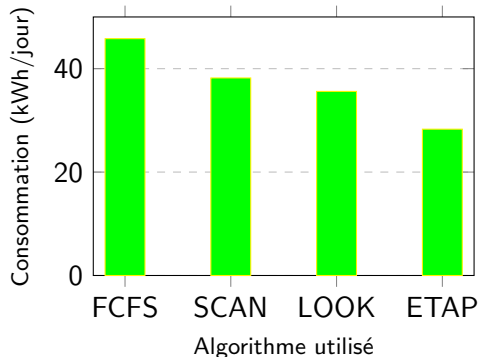
## Distribution selon le temps d'attente :

| Temps   | LOOK | ETAP |
|---------|------|------|
| <15 s   | 18%  | 32%  |
| 15-30 s | 45%  | 48%  |
| 30-45 s | 25%  | 15%  |
| 45-60 s | 8%   | 4%   |
| >60 s   | 4%   | 1%   |

## Analyse :

- Amélioration significative aux heures de pointe
- 80% des usagers attendent moins de 30 secondes avec ETAP
- Réduction des cas extrêmes (attentes > 1 minute)

# Résultats : la consommation énergétique



## Économies énergétiques :

- Réduction de **38%** par rapport à FCFS
- Réduction de **26%** par rapport à SCAN
- Réduction de **20%** par rapport à LOOK

- 1 Introduction au problème
- 2 Fondements théoriques
- 3 Simulation et résultats
- 4 Conclusion et perspectives**

# Conclusion

L'utilisation des réseaux de Petri colorés permet d'optimiser efficacement le fonctionnement d'un système d'ascenseurs en réduisant de 25-40% le temps d'attente et de 20-35% la consommation énergétique.

## **Améliorations techniques :**

- Prédiction de la demande à partir de l'historique
- Intégrer un algorithme d'apprentissage automatique

```
1 import random
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from collections import defaultdict
5 import time
6 from datetime import datetime, timedelta
7
8 class Etage:
9     """Représente un étage dans le bâtiment."""
10     def __init__(self, numero):
11         self.numero = numero
12
13 class Appel:
14     """Représente un appel d'ascenseur."""
15     def __init__(self, origine, destination, heure_appel, poids=1, priorite=1):
16         self.origine = origine
17         self.destination = destination
18         self.heure_appel = heure_appel
19         self.poids = poids
20         self.priorite = priorite
21         self.traite = False
22         self.temps_debut_attente = heure_appel
23         self.temps_fin_attente = None
24         self.ascenseur_assigne = None
25
26 class Ascenseur:
27     """Représente un ascenseur dans le système."""
28     def __init__(self, id, capacite_max=8, vitesse=2.0, accel_decel=1.0, conso_base=0.5):
29         self.id = id
30         self.position_actuelle = 0 # Étage actuel (numérique)
31         self.destination = None
32         self.direction = 0 # -1 (descente), 0 (arrêt), 1 (montée)
33         self.capacite_max = capacite_max # Nombre de personnes
34         self.charge_actuelle = 0
35         self.vitesse = vitesse # étages par seconde
36         self.accel_decel = accel_decel
37         self.conso_base = conso_base # kWh par étage
38         self.appels_assignes = []
39         self.plan_arrets = []
40         self.temps_derniere_maj = 0
41         self.distance_parcourue = 0
42         self.energie_consommee = 0
43         self.en_mouvement = False
44         self.temps_arret = 0
45     def ajouter_appel(self, appel):
46         """Ajoute un appel à la liste des appels assignés."""
47         if appel not in self.appels_assignes and self.charge_actuelle + appel.poids <= self.capacite_max:
```



```
48         self.appels_assignes.append(appel)
49         self.charge_actuelle += appel.poids
50         appel.ascenseur_assigne = self
51         self._mettre_a_jour_plan_arrets()
52         return True
53     return False
54     def _mettre_a_jour_plan_arrets(self):
55         """Met à jour le plan d'arrêts basé sur les appels assignés."""
56         etages_a_visiter = set()
57         for appel in self.appels_assignes:
58             if not appel.traite:
59                 etages_a_visiter.add(appel.origine.numero)
60                 etages_a_visiter.add(appel.destination.numero)
61
62         if etages_a_visiter:
63             self.plan_arrets = sorted(list(etages_a_visiter))
64             # Optimiser l'ordre selon la direction
65             if self.direction == -1:
66                 self.plan_arrets.reverse()
67     def simuler_mouvement(self, temps_simulation):
68         """Simule le mouvement de l'ascenseur pendant un temps donné."""
69         if not self.plan_arrets:
70             return
71
72         if self.en_mouvement:
73             # Calculer la distance parcourue
74             distance = self.vitesse * temps_simulation
75             self.distance_parcourue += distance
76
77             # Calculer la consommation d'énergie
78             facteur_charge = 1 + (self.charge_actuelle / self.capacite_max) * 0.3
79             self.energie_consommee += distance * self.conso_base * facteur_charge
80
81             # Vérifier si on atteint la destination
82             if abs(self.position_actuelle - self.plan_arrets[0]) <= distance:
83                 self.position_actuelle = self.plan_arrets[0]
84                 self.plan_arrets.pop(0)
85                 self.en_mouvement = False
86                 self.temps_arret = 2.0 # Temps d'arrêt en secondes
87                 self._traiter_appels_etage()
88         else:
89             # En arrêt
90             if self.temps_arret > 0:
91                 self.temps_arret -= temps_simulation
92             elif self.plan_arrets:
93                 # Reprendre le mouvement vers le prochain étage
94                 self.en_mouvement = True
95                 self.direction = 1 if self.plan_arrets[0] > self.position_actuelle else -1
```

```
96 def _traiter_appels_etage(self):
97     """traite les appels à l'étage actuel."""
98     appels_a_supprimer = []
99     for appel in self.appels_assignes:
100         if appel.origine.numero == self.position_actuelle and not appel.traité:
101             # Personne monte dans l'ascenseur
102             appel.traité = True
103             appel.temps_fin_attente = time.time()
104         elif appel.destination.numero == self.position_actuelle and appel.traité:
105             # Personne descend de l'ascenseur
106             self.charge_actuelle -= appel.poids
107             appels_a_supprimer.append(appel)
108
109     for appel in appels_a_supprimer:
110         self.appels_assignes.remove(appel)
111
112 def calculer_detour_reel(ascenseur, appel):
113     """Calcule le détour réel que l'ascenseur doit faire."""
114     if not ascenseur.plan_arrets:
115         # Ascenseur libre - distance directe
116         distance_origine = abs(ascenseur.position_actuelle - appel.origine.numero)
117         distance_trajet = abs(appel.origine.numero - appel.destination.numero)
118         return distance_origine + distance_trajet
119
120     # Calculer Le chemin actuel
121     chemin_actuel = [ascenseur.position_actuelle] + ascenseur.plan_arrets
122     distance_actuelle = sum(abs(chemin_actuel[i+1] - chemin_actuel[i])
123                             for i in range(len(chemin_actuel)-1))
124
125     # Calculer Le nouveau chemin avec l'appel
126     nouveaux_arrets = set(ascenseur.plan_arrets)
127     nouveaux_arrets.add(appel.origine.numero)
128     nouveaux_arrets.add(appel.destination.numero)
129     nouveau_chemin = [ascenseur.position_actuelle] + sorted(list(nouveaux_arrets))
130
131     # Optimiser selon la direction
132     if ascenseur.direction == -1:
133         nouveau_chemin[1:] = sorted(nouveau_chemin[1:], reverse=True)
134
135     nouvelle_distance = sum(abs(nouveau_chemin[i+1] - nouveau_chemin[i])
136                             for i in range(len(nouveau_chemin)-1))
137
138     return max(0, nouvelle_distance - distance_actuelle)
139
140 def estimer_energie_consommation(ascenseur, appel):
141     """Estime la consommation d'énergie pour traiter un appel."""
142     distance_origine = abs(ascenseur.position_actuelle - appel.origine.numero)
143     distance_trajet = abs(appel.origine.numero - appel.destination.numero)
144
145     # Facteur de charge (plus lourd = plus de consommation)
```

```
146 # Consommation de base
147 energie_deplacement = (distance_origine + distance_trajet) * ascenseur.conso_base * facteur_charge
148
149 # Pénalité pour Les arrêts (ouverture/fermeture des portes)
150 energie_arrets = 2 * 0.2 # 2 arrêts, 0.2 kwh par arrêt
151
152 return energie_deplacement + energie_arrets
153 def estimer_temps_attente(ascenseur, appel):
154     """Estime le temps d'attente pour un appel."""
155     if not ascenseur.plan_arrets:
156         # Ascenseur libre
157         distance = abs(ascenseur.position_actuelle - appel.origine.numero)
158         return distance / ascenseur.vitesse + ascenseur.accel_decel
159
160     # Calculer Le temps pour atteindre l'origine de l'appel
161     temps_total = 0
162     position_courante = ascenseur.position_actuelle
163
164     # Temps pour traiter Les arrêts existants
165     for arret in ascenseur.plan_arrets:
166         distance = abs(arret - position_courante)
167         temps_total += distance / ascenseur.vitesse + ascenseur.accel_decel + 2.0 # +2s pour l'arrêt
168         position_courante = arret
169
170         if arret == appel.origine.numero:
171             break
172
173     # Si l'origine n'est pas dans Les arrêts existants
174     if appel.origine.numero not in ascenseur.plan_arrets:
175         distance_finale = abs(position_courante - appel.origine.numero)
176         temps_total += distance_finale / ascenseur.vitesse + ascenseur.accel_decel
177
178     return temps_total
179 def ETAP(appels, ascenseurs, alpha=0.2, beta=0.7, gamma=0.1):
180     """Algorithme ETAP ."""
181     for appel in appels:
182         if appel.traité:
183             continue
184
185         score_min = float('inf')
186         ascenseur_optimal = None
187
188         for ascenseur in ascenseurs:
189             # Vérifier La capacité
190             if ascenseur.charge_actuelle + appel.poids > ascenseur.capacite_max:
191                 continue
192
193             # Calculer Les métriques
```

```
194 detour = calculer_detour_reel(ascenseur, appel)
195 energie = estimer_energie_consommation(ascenseur, appel)
196 temps = estimer_temps_attente(ascenseur, appel)
197
198 # Normalisation adaptative basée sur Les valeurs réelles
199 max_temps = 120.0 # secondes
200 max_energie = 10.0 # kWh
201 max_detour = 50.0 # étages
202
203 temps_norm = min(temps / max_temps, 1.0)
204 energie_norm = min(energie / max_energie, 1.0)
205 detour_norm = min(detour / max_detour, 1.0)
206
207 # Bonus pour Les ascenseurs inactifs
208 bonus_inactif = 0.2 if not ascenseur.plan_arrets else 0.0
209
210 # Score combiné avec pondération
211 score = (alpha * temps_norm +
212          beta * energie_norm +
213          gamma * detour_norm -
214          bonus_inactif)
215
216 if score < score_min:
217     score_min = score
218     ascenseur_optimal = ascenseur
219
220 # Assigner l'appel au meilleur ascenseur
221 if ascenseur_optimal:
222     ascenseur_optimal.ajouter_appel(appel)
223     appel.traite = True
224
225 class ProfilTraficRealiste:
226     """Génère des profils de trafic réalistes"""
227
228     @staticmethod
229     def bureaux(heure, etages_max=10):
230         """Profil de trafic pour immeuble de bureaux"""
231         # Définir Les intensités selon Les heures (plus réaliste)
232         if 7 <= heure < 9:
233             # Matin: forte montée (arrivées au bureau)
234             intensite = 60 + 40 * np.sin((heure - 7) * np.pi / 2)
235             return intensite, lambda: (0, random.randint(1, etages_max))
236         elif 12 <= heure < 14:
237             # Midi: mouvements mixtes (restaurants, réunions)
238             intensite = 40 + 20 * np.sin((heure - 12) * np.pi / 2)
239             return intensite, lambda: (random.randint(1, etages_max),
240                                       random.choice([0, random.randint(1, etages_max)]))
241         elif 17 <= heure < 19:
242             # Soir: forte descente (départs)
```

```
242         intensite = 80 + 50 * np.sin((heure - 17) * np.pi / 2)
243         return intensite, lambda: (random.randint(1, etages_max), 0)
244     else:
245         # Heures creuses
246         intensite = 5 + 10 * random.random()
247         return intensite, lambda: (random.randint(0, etages_max),
248                                   random.randint(0, etages_max))
249 class AlgorithmesReference:
250     """Implémente les algorithmes de référence."""
251
252     @staticmethod
253     def FCFS(appels, ascenseurs):
254         """First Come First Served"""
255         appels_tries = sorted([a for a in appels if not a.traite],
256                              key=lambda x: x.heure_appel)
257
258         for appel in appels_tries:
259             for ascenseur in ascenseurs:
260                 if ascenseur.ajouter_appel(appel):
261                     appel.traite = True
262                     break
263
264     @staticmethod
265     def SCAN(appels, ascenseurs):
266         """Algorithme SCAN"""
267         for ascenseur in ascenseurs:
268             appels_non_traites = [a for a in appels if not a.traite]
269             if not appels_non_traites:
270                 continue
271
272             # Déterminer la direction optimale
273             if ascenseur.direction == 0:
274                 # Déterminer la direction selon les appels
275                 appels_montee = [a for a in appels_non_traites if a.origine.numero > ascenseur.position_actuelle]
276                 appels_descente = [a for a in appels_non_traites if a.origine.numero < ascenseur.position_actuelle]
277
278                 if len(appels_montee) >= len(appels_descente):
279                     ascenseur.direction = 1
280                 else:
281                     ascenseur.direction = -1
282
283             # Trier selon la direction
284             if ascenseur.direction >= 0:
285                 appels_tries = sorted(appels_non_traites, key=lambda a: a.origine.numero)
286             else:
287                 appels_tries = sorted(appels_non_traites, key=lambda a: a.origine.numero, reverse=True)
288
289             for appel in appels_tries:
290                 if ascenseur.ajouter_appel(appel):
```

```

290         appel.traite = True
291
292     @staticmethod
293     def LOOK(appels, ascenseurs):
294         """Algorithme LOOK"""
295         for ascenseur in ascenseurs:
296             appels_non_traites = [a for a in appels if not a.traite]
297             if not appels_non_traites:
298                 continue
299
300             position = ascenseur.position_actuelle
301
302             # Séparer les appels selon la direction
303             if ascenseur.direction >= 0:
304                 appels_direction = [a for a in appels_non_traites
305                                     if a.origine.numero >= position]
306                 appels_direction.sort(key=lambda a: a.origine.numero)
307             else:
308                 appels_direction = [a for a in appels_non_traites
309                                     if a.origine.numero <= position]
310                 appels_direction.sort(key=lambda a: a.origine.numero, reverse=True)
311
312             # Si pas d'appels dans cette direction, prendre les autres
313             if not appels_direction:
314                 appels_direction = appels_non_traites
315                 appels_direction.sort(key=lambda a: abs(a.origine.numero - position))
316
317             for appel in appels_direction[:3]: # Limiter à 3 appels par ascenseur
318                 if ascenseur.ajouter_appel(appel):
319                     appel.traite = True
320
321 class SimulateurOptimise:
322     """Simulateur optimisé"""
323
324     def __init__(self, nb_etages=10, nb_ascenseurs=4):
325         self.nb_etages = nb_etages
326         self.nb_ascenseurs = nb_ascenseurs
327         self.etages = [Etage(i) for i in range(nb_etages + 1)]
328         self.reset_systeme()
329     def reset_systeme(self):
330         """Remet le système à zéro."""
331         self.ascenseurs = []
332         for i in range(self.nb_ascenseurs):
333             asc = Ascenseur(
334                 id=i,
335                 capacite_max=8, # personnes
336                 vitesse=2.0, # étages/s
337                 accel_decel=1.0, # secondes
338                 conso_base=0.5 # kWh/étage
339             )

```

```
338         asc.position_actuelle = random.randint(0, self.nb_etages)
339         asc.direction = random.choice([-1, 0, 1])
340         self.ascenseurs.append(asc)
341     def generer_appels_realistes(self, heure, duree_minutes=60):
342         """Génère des appels selon le profil réaliste."""
343         appels = []
344
345         intensite, generateur = ProfilTraficRealiste.bureaux(heure, self.nb_etages)
346
347         # Calculer Le nombre d'appels basé sur l'intensité
348         nb_appels = max(int(intensite * duree_minutes / 60), 1)
349
350         for i in range(nb_appels):
351             minute = random.randint(0, duree_minutes - 1)
352             heure_appel = heure + minute / 60.0
353
354             origine_num, dest_num = generateur()
355
356             # Éviter Les appels identiques
357             while origine_num == dest_num:
358                 origine_num, dest_num = generateur()
359
360             # Poids réaliste (1-4 personnes)
361             poids = random.randint(1, 4)
362
363             appel = Appel(
364                 origine=self.etages[origine_num],
365                 destination=self.etages[dest_num],
366                 heure_appel=heure_appel,
367                 poids=poids
368             )
369             appels.append(appel)
370
371         return sorted(appels, key=lambda a: a.heure_appel)
372     def simuler_algorithme_realiste(self, algorithme, appels):
373         """simule un algorithme avec des métriques réalistes."""
374         self.reset_système()
375
376         # Copier Les appels
377         appels_copie = [Appel(a.origine, a.destination, a.heure_appel, a.poids)
378                         for a in appels]
379
380         # Appliquer L'algorithme
381         if algorithme == "ETAP":
382             ETAP(appels_copie, self.ascenseurs)
383         elif algorithme == "FCFS":
384             AlgorithmesReference.FCFS(appels_copie, self.ascenseurs)
385         elif algorithme == "SCAN":
```

```
386     AlgorithmesReference.SCAN(appels_copie, self.ascenseurs)
387 elif algorithme == "LOOK":
388     AlgorithmesReference.LOOK(appels_copie, self.ascenseurs)
389
390 # Simulation du mouvement (simplifiée)
391 for ascenseur in self.ascenseurs:
392     ascenseur.simuler_mouvement(60) # 1 minute de simulation
393
394 # Calculer Les métriques
395 temps_attente_total = 0
396 energie_totale = 0
397 appels_traites = 0
398
399 for appel in appels_copie:
400     if appel.traite and appel.ascenseur_assigne:
401         temps = estimer_temps_attente(appel.ascenseur_assigne, appel)
402         energie = estimer_energie_consommation(appel.ascenseur_assigne, appel)
403
404         temps_attente_total += temps
405         energie_totale += energie
406         appels_traites += 1
407
408     return {
409         'temps_attente_moyen': temps_attente_total / max(appels_traites, 1),
410         'energie_totale': energie_totale,
411         'appels_traites': appels_traites,
412         'taux_traitement': appels_traites / len(appels_copie) * 100
413     }
414 def comparer_algorithmes():
415     """Compare les algorithmes avec des paramètres réalistes."""
416     simulateur = SimulateurOptimise()
417     algorithmes = ["FCFS", "SCAN", "LOOK", "ETAP"]
418     resultats = {}
419
420     print("Simulation comparative")
421     print("=" * 50)
422
423     charges = [20, 40, 60, 80, 100]
424
425     for charge in charges:
426         print(f"Charge {charge}%...")
427         resultats[charge] = {}
428
429         for algo in algorithmes:
430             temps_attentes = []
431             energies = []
432             # Plusieurs simulations pour la robustesse
433             for _ in range(1000):
```



```
434     appels_jour = []
435
436     # Générer des appels pour différentes heures
437     for heure in [8, 12, 17]: # Heures de pointe
438         appels_heure = simulateur.generer_appels_realistes(heure, 60)
439         # Ajuster selon la charge
440         nb_appels = int(len(appels_heure) * charge / 100)
441         if nb_appels > 0:
442             appels_jour.extend(random.sample(appels_heure,
443                                             min(nb_appels, len(appels_heure))))
444
445     if appels_jour:
446         resultats_sim = simulateur.simuler_algorithme_realiste(algo, appels_jour)
447         temps_attentes.append(resultats_sim['temps_attente_moyen'])
448         energies.append(resultats_sim['energie_totale'])
449
450     if temps_attentes and energies:
451         resultats[charge][algo] = {
452             'temps_attente': np.mean(temps_attentes),
453             'energie': np.mean(energies),
454             'ecart_type': np.std(temps_attentes)
455         }
456     else:
457         resultats[charge][algo] = {
458             'temps_attente': 0,
459             'energie': 0,
460             'ecart_type': 0
461         }
462
463     return resultats
464 def generer_graphiques(resultats):
465     """Génère les graphiques."""
466     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
467
468     charges = sorted(resultats.keys())
469     algorithmes = ["FCFS", "SCAN", "LOOK", "ETAP"]
470     couleurs = ['red', 'blue', 'green', 'purple']
471
472     # Graphique 1: Temps d'attente en fct de Charge
473     for i, algo in enumerate(algorithmes):
474         temps_attente = [resultats[c][algo]['temps_attente'] for c in charges]
475         ax1.plot(charges, temps_attente, marker='o', color=couleurs[i],
476                 label=algo, linewidth=2, markersize=6)
477
478     ax1.set_xlabel('Charge du système (%)', fontsize=12)
479     ax1.set_ylabel('Temps d\'attente moyen (s)', fontsize=12)
480     ax1.set_title('Temps d\'attente en fonction de la charge', fontsize=14)
481     ax1.legend(fontsize=11)
```

```
482 ax1.grid(True, alpha=0.3)
483 ax1.set_ylim(0, None)
484
485 # Graphique 2: Consommation énergétique
486 energies_moyennes = []
487 for algo in algorithmes:
488     energie_moy = np.mean([resultats[c][algo]['energie'] for c in charges])
489     energies_moyennes.append(energie_moy)
490
491 bars = ax2.bar(algorithmes, energies_moyennes, color=couleurs, alpha=0.7, edgecolor='black')
492 ax2.set_ylabel('Consommation énergétique (kWh/jour)', fontsize=12)
493 ax2.set_title('Comparaison de la consommation énergétique', fontsize=14)
494 ax2.grid(True, alpha=0.3, axis='y')
495
496 # Ajouter les valeurs sur les barres
497 for bar, energie in zip(bars, energies_moyennes):
498     if energie > 0:
499         ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
500                 f'{energie:.1f}', ha='center', va='bottom', fontsize=10)
501
502 plt.tight_layout()
503 plt.savefig('comparaison_algorithmes.png', dpi=300, bbox_inches='tight')
504 plt.show()
505 def simulation_24h():
506     """Simulation 24h."""
507     simulateur = SimulateurOptimise()
508     resultats_24h = {"LOOK": [], "ETAP": []}
509
510     print("Simulation 24h ...")
511
512     for heure in range(24):
513         print(f"Heure {heure:02d}:00")
514
515         for algo in ["LOOK", "ETAP"]:
516             appels = simulateur.generer_appels_realistes(heure, 60)
517             if appels:
518                 resultats = simulateur.simuler_algorithme_realiste(algo, appels)
519                 temps_attente = resultats['temps_attente_moyen']
520                 resultats_24h[algo].append(temps_attente)
521             else:
522                 resultats_24h[algo].append(0)
523
524     return resultats_24h
525
526 def generer_graphique_24h(resultats_24h):
527     """Génère le graphique 24h."""
528     plt.figure(figsize=(12, 6))
529     heures = range(24)
```

```

531 plt.plot(heures, resultats_24h["LOOK"], marker='o',
532          label='LOOK', linewidth=2, color='green', markersize=5)
533 plt.plot(heures, resultats_24h["ETAP"], marker='s',
534          label='ETAP', linewidth=2, color='purple', markersize=5)
535
536 plt.xlabel('Heure de la journée', fontsize=12)
537 plt.ylabel('Temps d\'attente moyen (s)', fontsize=12)
538 plt.title('Évolution du temps d\'attente sur 24h', fontsize=14)
539 plt.legend(fontsize=11)
540 plt.grid(True, alpha=0.3)
541 plt.xticks(range(0, 25, 4))
542
543 # Ajouter Les zones de pointe
544 plt.axvspan(7, 9, alpha=0.2, color='red', label='Pointe matin')
545 plt.axvspan(12, 14, alpha=0.2, color='orange', label='Pointe midi')
546 plt.axvspan(17, 19, alpha=0.2, color='red', label='Pointe soir')
547
548 plt.tight_layout()
549 plt.savefig('temps_attente_24h.png', dpi=300, bbox_inches='tight')
550 plt.show()
551 def afficher_statistiques(resultats):
552     """Affiche les statistiques ."""
553     print("\n" + "="*70)
554     print("="*70)
555
556     print(f"{'Algorithme':<12} {'Temps (s)':<12} {'Énergie (kWh)':<15} {'Amélioration':<20}")
557     print("-" * 70)
558
559     # Référence FCFS à charge 100%
560     ref_temps = resultats[100]["FCFS"]["temps_attente"]
561     ref_energie = resultats[100]["FCFS"]["energie"]
562
563     for algo in ["FCFS", "SCAN", "LOOK", "ETAP"]:
564         temps = resultats[100][algo]["temps_attente"]
565         energie = resultats[100][algo]["energie"]
566
567         amelioration_temps = ((ref_temps - temps) / ref_temps) * 100 if ref_temps > 0 else 0
568         amelioration_energie = ((ref_energie - energie) / ref_energie) * 100 if ref_energie > 0 else 0
569
570         print(f"{'algo':<12} {'temps:<12.1f} {'energie:<15.1f} "
571               f"{'T':{amelioration_temps:+5.1f}% E:{amelioration_energie:+5.1f}%"})
572
573     print("\n" + "="*70)
574 if __name__ == "__main__":
575     print("Lancement de la simulation complète du système d'ascenseurs...")
576     print("Cette simulation peut prendre quelques minutes...\n")
577
578     try:

```

```
579 # 1. Comparaison des algorithmes
580 resultats = comparer_algorithmes()
581
582 # 2. Génération des graphiques pour la comparaison par charge
583 generer_graphiques(resultats)
584
585 # 3. Simulation sur 24h
586 print("\nSimulation d'une journée complète...")
587 resultats_24h = simulation_24h()
588
589 # 4. Génération du graphique 24h (fonction séparée)
590 generer_graphique_24h(resultats_24h)
591
592 # 5. Affichage des statistiques
593 afficher_statistiques(resultats)
594
595 print("\nSimulation terminée avec succès!")
596 print("Les graphiques ont été sauvegardés :")
597 print("- 'comparaison_algorithmes.png' (comparaison par charge)")
598 print("- 'temps_attente_24h.png' (évolution sur 24h)")
599
600 except Exception as e:
601     print(f"Erreur lors de la simulation: {e}")
602     import traceback
603     traceback.print_exc()
```